

Design Pattern

1. Singleton Design Pattern

1.1. Singleton Design Pattern Scenario

A university system needs to maintain only one university information object containing the university name and address. All employees such as teachers, officers, and staff need to access the same information. Creating multiple university objects may cause inconsistency, so the Singleton Pattern is used to ensure only one instance of the University class exists. The class diagram contains the University class with a private instance variable and a getInstance() method that returns the single object.

Defination:

The Singleton Design Pattern is a creational design pattern that ensures:

1. A class has only one object (instance) throughout the entire application.
2. That single object provides a global access point to its data and methods.

In simple words: "Singleton allows only one instance of a class to exist and prevents creating multiple objects of that class."

In your scenario, the University class is implemented as a Singleton.

The university information:

- University name
- University address

is common for all employees. There is no need to create separate university objects for teachers, officers, and staff.

1.2. Why Do We Use Singleton Pattern?

Singleton is used when only one shared object should exist.

1. Avoid duplicate objects

Without Singleton:

Teacher ---> University Object 1

Officer ---> University Object 2

Staff ---> University Object 3

Multiple university objects waste memory.

With Singleton:

Teacher

|

Officer

|

Staff

|

v

Single University Object

2. Centralized information management

All users access the same data.

Example:

University Name:

Jashore University of Science and Technology

Address:

Ambottola, Jashore

Every employee sees the same information.

3. Maintain consistency

If one object changes the data, everyone gets the updated data.

Example:

University address changed

Singleton Object Updated

Teacher sees new address

Officer sees new address

Staff sees new address

1.3. How to Identify Singleton Requirement from a Scenario?

After reading a scenario, ask these questions:

Question 1: "Should there be only one object of this class in the whole system?"

If yes → Singleton may be needed.

Example: A university system should have only one university information object.

Answer: University ---> Singleton

Question 2: "Do all parts of the system need the same shared information?"

If yes → Singleton is suitable.

Example:

Employee, Teacher, Officer, Staff

All need University Information

Question 3: "Will creating multiple objects create inconsistency?"

Example: Without Singleton:

University Object 1:

Address = Jashore

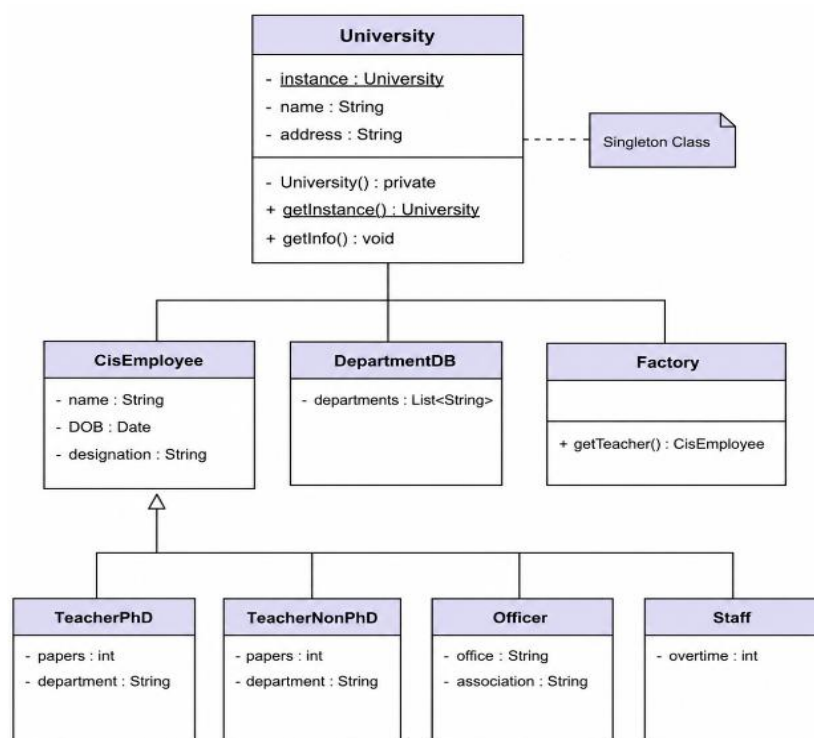
University Object 2:

Address = Khulna

Which one is correct?

Singleton avoids this problem.

1.4: UML Diagram



1.5: Pseudocode of Singleton Pattern

Class University:

private static instance = NULL

private name

private address

Function CreateObject():

If instance is NULL:

Create new University object

Store object in instance

Set name

Set address

Return instance

Function getUniversityInfo():

Display name

Display address

1.6. Which Design Patterns Can Replace Singleton?

Singleton can sometimes be replaced by:

1. Dependency Injection

Instead of creating:

University()

everywhere:

we pass the object:

Employee

|

v

University Object

Advantage:

- Easier testing
- Less dependency

2. Factory Pattern

Factory can control object creation.

Example:

Factory

|

+---- Create University Object

However, Factory does **not guarantee only one object** unless we add Singleton behavior.

3. Abstract Factory Pattern

Can replace Singleton when we need families of objects.

Example: UniversityFactory

Creates:

Teacher

Officer

Staff
But it is not a direct replacement.

1.7. Which Patterns Can Singleton Replace?

Singleton can replace:

Factory Pattern (sometimes)

Example:

Instead of:

Create Factory object every time

we make:

One Factory object using Singleton.

Builder Pattern (rare case)

A configuration builder can sometimes be Singleton.

Example: Application Configuration Builder

Abstract Factory (rare case)

A single factory provider can be Singleton.

Example: One Database Factory

2 Factory Design Pattern

2.1 A university recruitment system creates different types of teachers based on qualification. For example, PhD qualification creates a TeacherPhD object, and MSc qualification creates a TeacherNonPhD object. Instead of creating objects directly, the client uses a TeacherFactory that decides which object should be created. The class diagram contains a Factory class, an Employee interface, and different teacher classes.

Definition:

The Factory Design Pattern is a creational design pattern that provides an interface for creating objects but allows subclasses or a separate factory class to decide which class object should be created.

In simple words: "Factory Pattern creates objects without exposing the object creation logic to the client." The client asks the factory for an object, and the factory decides which concrete class should be instantiated. Factory Method pattern does not require an abstract class. Whether there is an abstract class depends on which factory pattern you're implementing.

2.2 Why Do We Use Factory Pattern?

Normally, without Factory Pattern:

```
teacher = clsTeacherPhD()
```

The client directly knows:

- Which class to create
- How to create the object
- Which constructor to call

This creates tight coupling.

If tomorrow we add:

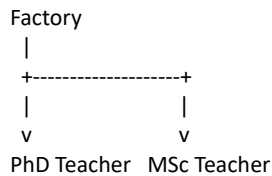
```
clsTeacherProfessor()
```

```
clsTeacherResearcher()
```

we need to modify the client code.

With Factory Pattern:

```
Client
|
|
v
```



The client only knows: `factory.get_teacher("PhD")`

The factory handles object creation.

2.3. Factory Pattern Components

1. Product Interface / Abstract Class

Defines common behavior.

Your example: `class clsEmployee(ABC):`

It defines: `write_info()`

2. Concrete Products

Actual classes created by the factory.

Your example:

`clsTeacherPhD`

`clsTeacherNonPhD`

Both inherit:

`clsEmployee`

3. Factory Class

Responsible for object creation.

Your example: `class clsFacultyFactory:`

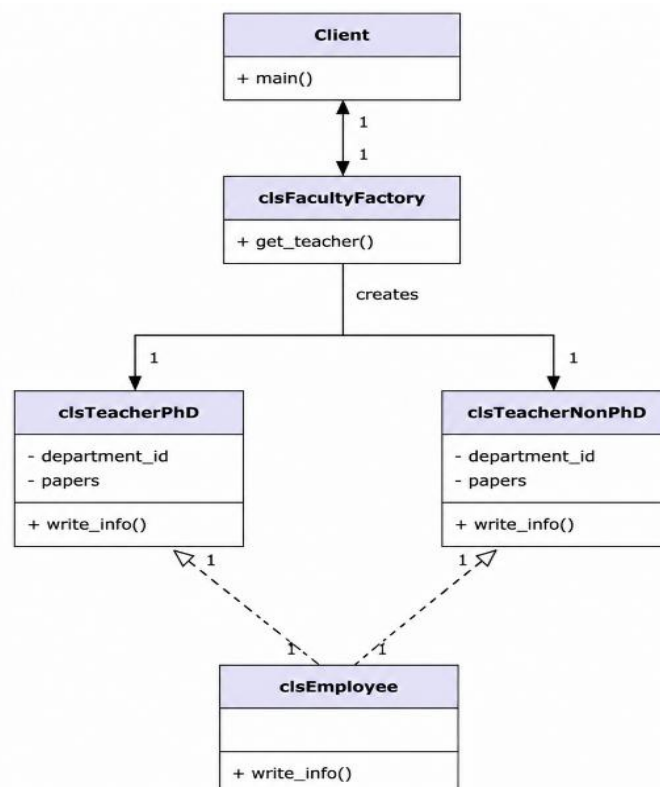
Method: `get_teacher()`

4. Client

Uses the objects.

Your example: `main()`

2.4 UML Diagram



2.5:Pseudocode

```
Class Factory:
    Function createTeacher(type):

        If type is "PhD":
            Create TeacherPhD object
            Return object

        Else If type is "MSc":
            Create TeacherNonPhD object
            Return object

        Else:
            Return NULL

Main:
    Create Factory object

    teacher1 = factory.createTeacher("PhD")
    teacher2 = factory.createTeacher("MSc")

    teacher1.write_info()
    teacher2.write_info()
```

2.6. How to Identify Factory Pattern from a Scenario?

Look for these situations:

1. Many classes implement the same interface

Example:

```
Employee
|
|---- Teacher
|---- Officer
|---- Staff
Different objects, same common behavior.
```

2. Object creation decision depends on input

Example:

```
Qualification = PhD
|
v
Create PhD Teacher

Qualification = MSc
|
v
Create Non-PhD Teacher
```

3. Client should not know concrete classes

Scenario: A university system creates teachers based on qualification. The user only provides qualification; the system decides which teacher object to create.

Factory is suitable.

4. New classes may be added later

Example:

Today:

PhD Teacher

MSc Teacher

Tomorrow:

Professor

Lecturer

Researcher

Only factory changes, client code remains unchanged.

2.7 Replacement of Factory Pattern by Other Design Patterns

Factory Pattern is mainly used for object creation, so it can be replaced by other creational patterns when the scenario changes.

1. Abstract Factory Pattern can replace Factory Pattern

Most common replacement.

Factory Pattern:

Creates objects from one category.

Example:

TeacherFactory

|

+---- TeacherPhD

+---- TeacherNonPhD

Abstract Factory:

Creates a family of related objects.

Example:

UniversityFactory

|

+---- TeacherFactory

|

+---- DepartmentFactory

|

+---- CourseFactory

If the system becomes larger and needs multiple related objects, Abstract Factory can replace Factory Pattern.

2. Builder Pattern can replace Factory Pattern

Builder is useful when object creation becomes complex.

Factory:

TeacherFactory.createTeacher("PhD")

Only decides **which class to create**.

Builder:

TeacherBuilder

setName()

setDepartment()

setQualification()

setPapers()

build()

If an object has many parameters, Builder can replace Factory.

Example:

A teacher object needs:

- Name
- Department
- Qualification
- Research papers
- Address
- Salary

Instead of:

Teacher(name, dept, qualification, papers, address, salary)

Builder is better.

Patterns That Factory Pattern Can Replace

1. Factory Pattern can replace direct object creation

Without Factory:

```
teacher = clsTeacherPhD()
```

The client knows the concrete class.

With Factory:

```
teacher = factory.get_teacher("PhD")
```

The factory hides the creation process.

2. Factory Pattern can replace Singleton (partially)

Factory can manage object creation, but it does **not naturally guarantee one object**.

Example:

Factory:

UniversityFactory

```
createUniversity()
```

can create:

University Object 1

University Object 2

If we modify the factory:

Factory

|

checks existing object

|

returns same object

then it behaves like Singleton.

However, Singleton is still the correct pattern when only one instance is required.

3. Factory Pattern can replace Simple Factory logic

Simple object creation:

```
if type=="A":
```

```
    object=A()
```

```
elif type=="B":
```

```
    object=B()
```

can be moved into a Factory:

Factory

```
|  
+-- create A  
+-- create B
```

3 Abstract Factory Design Pattern

3.1 A university system needs to create groups of related objects, such as teachers, officers, and staff for different university types. A Science University Factory creates Science employees, and an Engineering University Factory creates Engineering employees. The **Abstract Factory Pattern** provides an interface for creating these related object families without exposing their concrete classes

Defiation:

The Abstract Factory Design Pattern is a creational design pattern that provides an interface for creating families of related objects without specifying their concrete classes.

In simple words: "Abstract Factory creates groups of related objects, while hiding the actual object creation process from the client." It is called a Factory of Factories because one abstract factory can create multiple related factories.

3.2. Why Do We Use Abstract Factory Pattern?

Suppose a university system needs to create different types of objects.

Without Abstract Factory:

```
Client  
|  
+-- Create Teacher  
|  
+-- Create Officer  
|  
+-- Create Staff  
|  
+-- Create Department
```

The client knows every concrete class.

Problems:

- Tight coupling
- Difficult to maintain
- Adding new object families requires many changes

With Abstract Factory:

```
Client  
|  
v  
+-----+  
| UniversityFactory |  
+-----+  
|  
+-----+  
|  
+-----+  
v          v  
TeacherFactory  StaffFactory  
|              |  
v              v
```

Teacher Objects Staff Objects

The client only communicates with the factory interface.

3.3. Components of Abstract Factory Pattern

1. Abstract Factory

Defines methods to create objects.

Example: UniversityFactory

Methods:

createTeacher()

createOfficer()

createStaff()

2. Concrete Factory

Implements object creation.

Example: ScienceFacultyFactory

EngineeringFacultyFactory

3. Abstract Products

Interfaces for products.

Example:

Teacher

Officer

Staff

4. Concrete Products

Actual objects.

Example:

TeacherPhD

TeacherNonPhD

Officer

Staff

5. Client

Uses factories and products.

Example: main()

3.4. Pseudocode of Abstract Factory Pattern

Interface UniversityFactory:

createTeacher()

createOfficer()

createStaff()

Class ScienceUniversityFactory implements UniversityFactory:

Function createTeacher():

return ScienceTeacher

```
Function createOfficer():  
    return ScienceOfficer
```

```
Function createStaff():  
    return ScienceStaff
```

Class EngineeringUniversityFactory implements UniversityFactory:

```
Function createTeacher():  
    return EngineeringTeacher
```

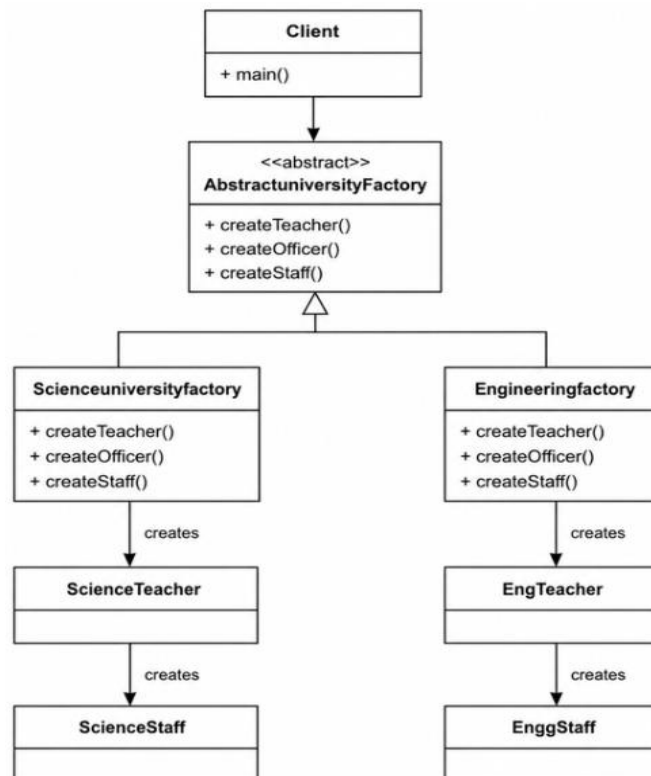
```
Function createOfficer():  
    return EngineeringOfficer
```

```
Function createStaff():  
    return EngineeringStaff
```

Main:

```
factory = new ScienceUniversityFactory  
teacher = factory.createTeacher()  
officer = factory.createOfficer()  
  
staff = factory.createStaff()  
display information
```

3.5 UML class diagram



3.6. How to Identify Abstract Factory Pattern from a Scenario?

Ask these questions:

1. Does the system create multiple related objects?

Example: University System

Teacher

Officer

Staff

All belong to the employee family.

Answer: Abstract Factory may be needed.

2. Are there different families of objects?

Example: Public University:

Teacher

Officer

Staff

Private University:

Teacher

Officer

Staff

Both have the same types but different implementations.

3. Does the client need to switch between object families?

Example:

Choose:

Science Faculty Factory

OR

Engineering Faculty Factory

without changing client code.

3.7. Replacement Points

Patterns that can replace Abstract Factory:

1. Factory Pattern

If only one product family is needed.

Example:

Instead of: UniversityFactory

Teacher

Officer

Staff

use: TeacherFactory

2. Builder Pattern

If the main problem is complex object construction.

Example:

Creating a detailed teacher object:

TeacherBuilder

Name

Department

Qualification

Salary

Address

Abstract Factory can replace:

1. Factory Pattern

When the system grows.

Example:

Factory:

TeacherFactory

|

+-- TeacherPhD

+-- TeacherMSc

can become:

UniversityFactory

|

+-- TeacherFactory

+-- StaffFactory

+-- OfficerFactory

2. Multiple Simple Factories

Instead of:

TeacherFactory

OfficerFactory

StaffFactory

use: One Abstract Factory
that manages all related factories.

4. Builder Design Pattern

4.1 A university system needs to create teacher objects with many details such as name, date of birth, address, phone, designation, department, and research papers. Passing all these values through a large constructor makes object creation complex and difficult to manage. Therefore, the **Builder Pattern** is used to create a teacher object step by step. The TeacherBuilder sets required attributes one by one and finally creates the complete TeacherPhD or TeacherNonPhD object.

Definition

The Builder Design Pattern is a creational design pattern used to create complex objects step by step. Instead of creating an object using a constructor with many parameters, Builder separates the object construction process from the final object representation.

In simple words: "Builder Pattern builds a complex object gradually by setting different parts and then creating the final object."

Example:

A Teacher object contains:

- Name
- Date of birth
- Address
- Phone
- Designation
- Department

- Research papers

Creating it directly:

Teacher(name, dob, address, phone, designation, department, papers)
becomes difficult.

Builder allows:

- Set name
- Set address
- Set department
- Create teacher

4.2 Why Do We Use Builder Pattern?

1. Avoid Large Constructors

Without Builder:

```
Teacher(
name,
dob,
sex,
p_address,
c_address,
phone,
designation,
department,
papers
)
```

Problems:

- Difficult to remember parameter order.
- Hard to read.
- Error-prone.

Builder makes it clear:

```
setName()
setAddress()
setDepartment()
build()
```

2 Step-by-Step Object Creation

Some objects require multiple steps.

Example:

Teacher creation:

Basic Information

|

v

Academic Information

|

v

Department Information

|

v

Complete Teacher Object

3. Handle Optional Attributes

Some attributes may not always be required.

Example:

Teacher may have:

- Phone number
- Current address
- Research papers

Builder allows adding only required information.

5.3. How to Identify Builder Pattern from a Scenario?

Question 1: Does the object have many attributes?

Example:

Teacher:

Name

DOB

Address

Phone

Department

Papers

Salary

If yes → Builder may be needed.

Question 2: Is object creation complicated?

Example:

Create Employee:

Step 1: Personal information

Step 2: Academic information

Step 3: Department information

Step 4: Final object

Builder is suitable.

Question 3: Are many parameters optional?

Example: Teacher may have:

Phone

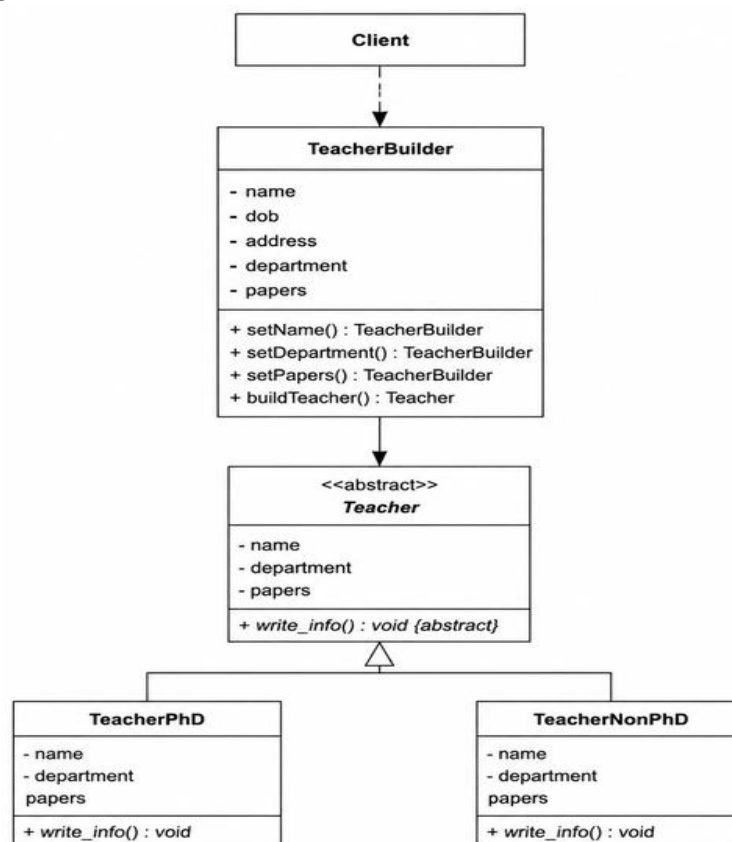
Current Address

Research Papers

Some may be missing.

Builder handles this easily.

4.4 UML Class Diagram for Scenario



4.5 Pseudocode

Class Teacher:

```
name
department
papers
```

Class TeacherBuilder:

```
name
department
papers
```

Function setName(name):

```
this.name = name
Return this
```

Function setDepartment(department):

```
this.department = department
Return this
```

Function setPapers(papers):

```
this.papers = papers
Return this
```

Function buildTeacher():

```
Return new Teacher(
    name,
    department,
    papers
)
```

Main:

```
builder = new TeacherBuilder()
teacher = builder
    .setName("Arif")
    .setDepartment("CSE")
    .setPapers(5)
    .buildTeacher()
```

```
teacher.displayInfo()
```

4.6 Which Design Patterns Can Replace Builder Pattern?

1. Factory Pattern (Partial Replacement)

Factory can replace Builder when object creation is simple.

Example:

Factory:

```
createTeacher("PhD")
```

Good when:

- Only object type selection is needed.
- Few parameters exist.

But Factory is not suitable for complex construction.

2. Abstract Factory Pattern (Partial Replacement)

Abstract Factory can replace Builder when the problem is creating different families of objects.

Example:

ScienceUniversityFactory

creates:

ScienceTeacher

ScienceOfficer

ScienceStaff

It focuses on **object families**, not step-by-step creation.

4.7 Which Patterns Can Builder Replace?

1. Builder can replace Factory (sometimes)

When Factory creation becomes complex.

Factory:

createTeacher()

Only decides which object.

Builder:

setName()

setDepartment()

setSalary()

build()

Handles detailed construction.

2. Builder can replace Abstract Factory (rare)

If the main requirement is constructing complex objects instead of creating object families.

5. Adapter Design Pattern

5.1 A university introduces a new employee system but still uses old HR systems. The old system provides data through different methods like `getTeachers()` and `getOfficers()`, while the new system requires `getEmployees()`. The Adapter Pattern converts the old interface into the new required interface without modifying the old system. The class diagram contains the Adapter, Target Interface, and Legacy classes.

Definition: The Adapter Design Pattern is a structural design pattern that allows two incompatible interfaces to work together by converting the interface of one class into another interface expected by the client.

In simple words: Adapter acts as a bridge between an existing class (legacy system) and a new system by changing the interface without modifying the original code. It is also known as the Wrapper Pattern.

5.2. Why Do We Use Adapter Pattern?

In real-world systems, we often need to integrate:

- Old systems (legacy systems)
- Third-party libraries
- External APIs
- New applications

But their interfaces may be different.

Example: Your university system expects:

`id="x1"`

```
employee.get_employees()
```

But the old HR system provides:

```
id="x2"
```

```
legacy.get_teachers()
```

The method names and data format are different. The old system cannot directly work with the new system. Adapter solves this problem.

5.3. Problem Without Adapter Pattern

Without Adapter:

New Employee System

|

|

X

Legacy HR System

The client expects:

```
id="x3"
```

```
get_employees()
```

but the legacy system provides:

```
id="x4"
```

```
get_teachers()
```

```
get_officers()
```

Problems:

- Need to modify legacy code.
- Creates dependency between systems.
- Difficult to maintain.

5.4. Solution Using Adapter Pattern

Adapter converts the old interface into the new interface.

Legacy Teaching HR System

|

|

v

+-----+

| Teacher Adapter |

+-----+

|

|

v

Employee System

```
get_employees()
```

Now the client does not care about the old system.

5.5 Pseudocode of Adapter Pattern

Interface EmployeeAdapter:

```
Function getEmployees()
```

Class LegacyTeachingHRSystem:

```
Function getTeachers()
```

```
Return teacher data
```

Class LegacyOfficialHRSystem:

```
Function getOfficers()
```

```
Return officer data
```

Class TeacherAdapter implements EmployeeAdapter:

```
Function getEmployees():  
    teachers = LegacyTeachingHRSysyem.getTeachers()  
    Convert data to Teacher objects  
    Return teachers
```

Class OfficerAdapter implements EmployeeAdapter:

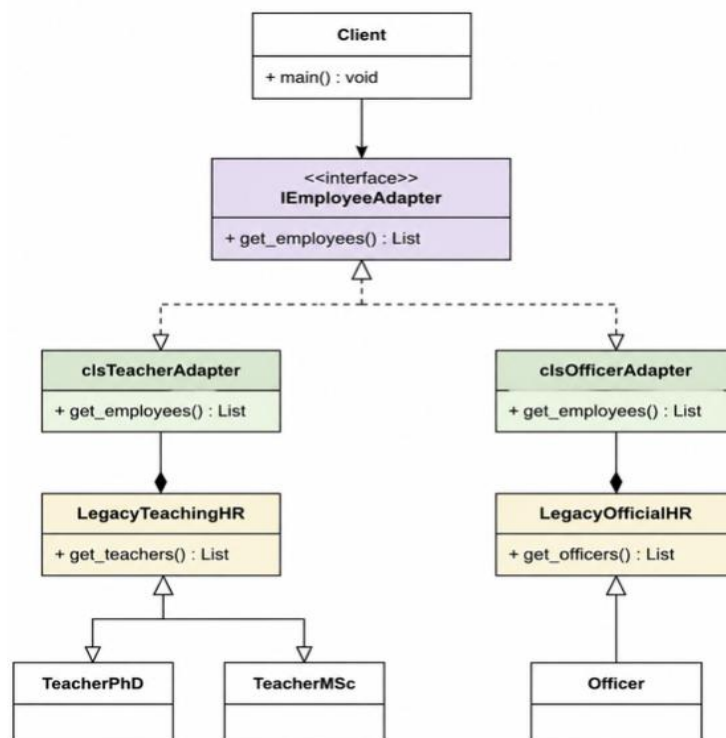
```
Function getEmployees():  
    officers = LegacyOfficialHRSysyem.getOfficers()  
    Convert data to Officer objects  
    Return officers
```

Main:

```
teacherAdapter = new TeacherAdapter()  
teacherList = teacherAdapter.getEmployees()  
Display teacherList
```

```
officerAdapter = new OfficerAdapter()  
officerList = officerAdapter.getEmployees()  
Display officerList
```

5.6 UML Class diagram



5.7 How to Identify Adapter Pattern from a Scenario?

Ask these questions:

1. Is there an existing class/system that cannot be modified?

Example: University already has an old HR system.

Answer: Adapter may be needed.

2. Does the old system provide different methods or data format?

Example:

Old system:

get_teachers()

New system:

get_employees()

Adapter converts them.

3. Do two systems need to communicate but have incompatible interfaces?

Example:

New Employee System

X

Legacy HR System

Adapter creates compatibility.

5.8 Adapter Pattern Replacement Points

Patterns That Can Replace Adapter

1. Decorator Pattern (partially)

Decorator can wrap objects and add behavior.

Example:

Employee

|

Decorator

|

Extra Features

But it does not solve interface mismatch.

Patterns Adapter Can Replace

1. Adapter can replace direct modification of legacy code

Instead of:

Modify Old HR System

use:

Old HR System

|

Adapter

|

New System

2. Adapter can replace manual conversion code

Without Adapter:

data = legacy.get_teachers()

convert manually

create employee objects

With Adapter:

adapter.get_employees()

6. Decorator Design Pattern

6.1 A pizza ordering system allows customers to customize pizzas by adding optional toppings such as extra cheese, mushroom, jalapeno, extra meat, and extra shrimp. Since customers can choose any combination of toppings, creating a separate class for every possible combination would make the system complex and difficult to maintain. Therefore, the Decorator Pattern is used. A basic pizza object is created first, and then different topping decorators are wrapped around it to add extra cost and functionality dynamically without modifying the original pizza classes.

Definition:

The Decorator Design Pattern is a structural design pattern that lets you dynamically add behavior to individual objects without changing other objects of the same class. It uses decorator classes to wrap concrete components, making functionality more flexible and reusable

In simple words:

Decorator Pattern wraps an existing object with another object to add extra features at runtime.

In your example:

- **Base Object:** Pizza
- **Decorators:** Extra Cheese, Mushroom, Jalapeno, Extra Meat, Extra Shrimp

Instead of creating separate classes like:

MargheritaWithCheese

MargheritaWithCheeseAndMushroom

MargheritaWithCheeseAndMushroomAndJalapeno

Decorator adds toppings dynamically.

6.2. Why Do We Use Decorator Pattern?

1. Add Features Dynamically

Instead of modifying the Pizza class, toppings are added when needed.

2. Avoid Huge Number of Classes

Without Decorator:

Pizza

MargheritaWithCheese

MargheritaWithCheeseAndMushroom

MargheritaWithCheeseAndMushroomAndShrimp

...

Too many classes.

Decorator uses only:

Pizza

ExtraCheese

Mushroom

Shrimp

3. Follow Open/Closed Principle

Open for extension.

Closed for modification.

We add new toppings without changing existing pizza classes.

4. Reuse Existing Classes

The same topping works for every pizza.

6.3 How to Identify Decorator Pattern?

Ask these questions:

Question 1: Can extra features be added after object creation?

Example:

Pizza

↓

Add Cheese

↓

Add Mushroom

↓

Add Shrimp

If yes → Decorator.

Question 2: Are there many combinations of optional features?

Example:

Pizza

Cheese

Cheese + Mushroom

Cheese + Mushroom + Shrimp

Decorator is appropriate.

Question 3: Do we want to avoid creating many subclasses?

Instead of:

PizzaWithCheese

PizzaWithCheeseAndMeat

PizzaWithCheeseAndShrimp

Decorator creates these combinations dynamically.

6.4 Pseudocode

Class BasePizza:

 getPrice()

Class Margherita inherits BasePizza:

 getPrice()

 Return base price

Class Gourmet inherits BasePizza:

 getPrice()

 Return base price

Class Topping inherits BasePizza:

 Store Pizza object

 getPrice()

 Return Pizza price + topping price

Class ExtraCheese inherits Topping

 Add cheese price

Class Mushroom inherits Topping

 Add mushroom price

Main

 pizza = Margherita()

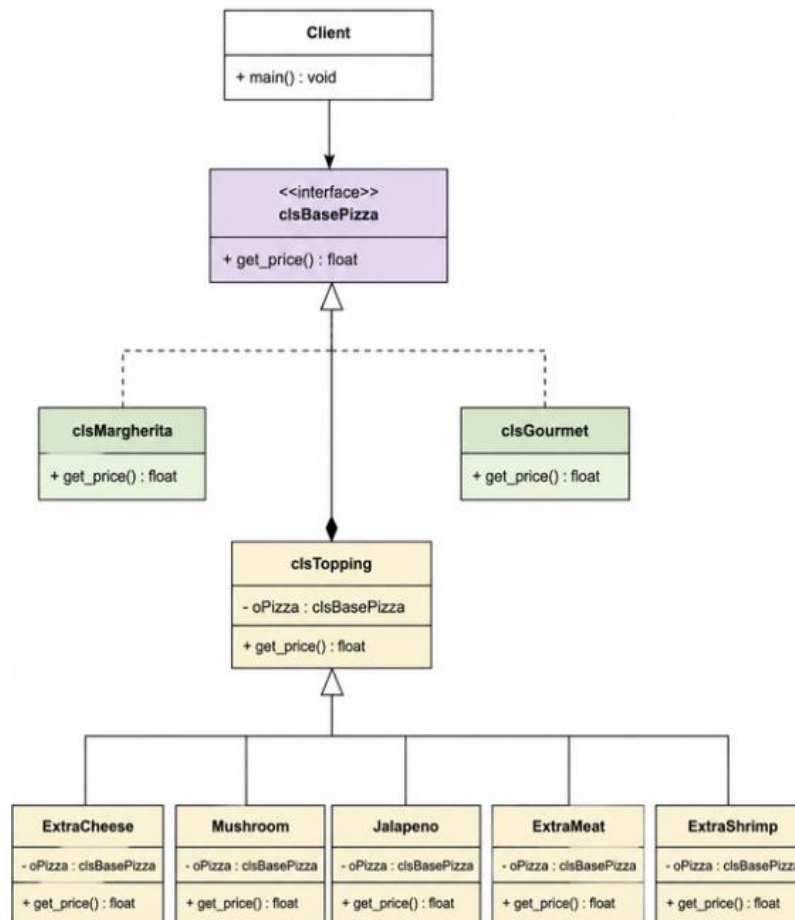
 pizza = ExtraCheese(pizza)

```

pizza = Mushroom(pizza)
pizza = ExtraMeat(pizza)
Display pizza.getPrice()

```

6.5 UML Class Diagram



6.6 Replacement

Patterns that can replace Decorator

1. Inheritance (Partial)

Instead of decorators:

Pizza

↓

PizzaWithCheese

↓

PizzaWithCheeseAndMushroom

Works only for a few combinations.

2. Composite (Sometimes)

If decorations form a tree structure, Composite may be used.

Decorator can replace

1. Inheritance

Decorator removes the need for many subclasses.

Instead of:

MargheritaWithCheese

MargheritaWithCheeseAndMushroom

MargheritaWithCheeseAndShrimp

Use:
Margherita
↓
Extra Cheese
↓
Mushroom
↓
Shrimp

7.Chain of Responsibility

7.1. What is Chain of Responsibility Pattern?

The Chain of Responsibility Pattern is a behavioral design pattern that allows a request to be passed through a chain of handlers until one handler can process the request.

In simple words: A request is sent to the first handler, and if it cannot handle the request, it forwards the request to the next handler in the chain.

In your scenario:

- Chairman approves small purchases.
- Treasurer approves medium purchases.
- Vice Chancellor approves larger purchases.
- Very large purchases go to Regent Board.

The requester does not need to know who will approve the purchase.

7.2. Scenario

A university purchase management system needs approval for different purchase amounts. Small purchases up to 25,000 can be approved by the Chairman. Purchases above 25,000 and up to 500,000 are forwarded to the Treasurer. Purchases above 500,000 and up to 1,000,000 are forwarded to the Vice Chancellor. If the amount is higher than that, it requires a Regent Board meeting. Instead of using many if-else statements in one class, the Chain of Responsibility Pattern creates a chain of approvers where each approver decides whether to approve or pass the request to the next authority.

7.3. Why Do We Use Chain of Responsibility?

1. Avoid Large Conditional Statements

Without Chain:

```
if amount < 25000:  
    chairman  
elif amount < 500000:  
    treasurer  
elif amount < 1000000:  
    vice_chancellor
```

This becomes difficult when new approval levels are added.

2. Reduce Coupling

The sender does not know:

- Who will handle the request.
- How approval decisions are made.

3. Easy to Add New Handlers

Example:

Add:

Dean Approval

between Treasurer and Vice Chancellor without changing existing classes.

4. Flexible Request Handling

Different requests can follow different chains.

7.4 How to Identify Chain of Responsibility?

Look for these situations:

Question 1: Are there multiple handlers with different responsibilities?

Example:

Chairman

Treasurer

Vice Chancellor

Question 2: Does the request need to move from one person/system to another?

Example:

Low amount → Chairman

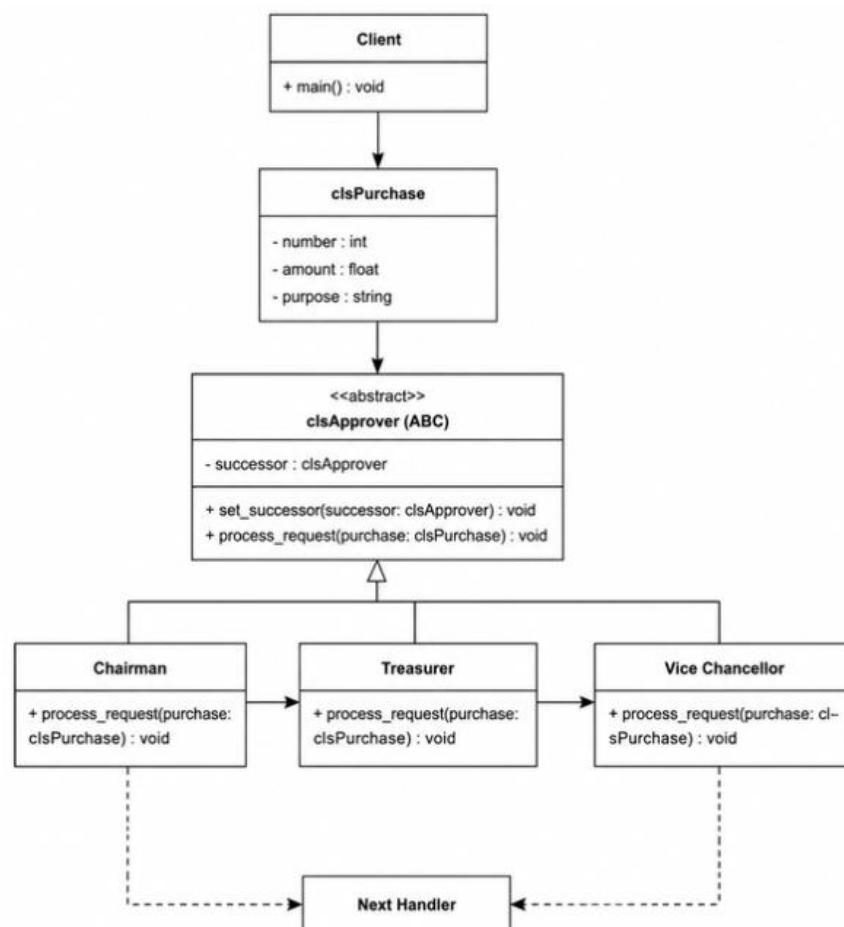
Medium amount → Treasurer

High amount → Vice Chancellor

Question 3: Does the sender not know the exact receiver?

Example: Employee submits a purchase request, but does not know which authority will approve it.

7.5 UML Class Diagram



7.6 Pseudocode

Class Purchase:

- Number
- amount
- purpose

Abstract Class Approver:

- successor
- setSuccessor(next)
- processRequest()

Class Chairman:

- If amount <= 25000:
 - Approve
- Else:
 - successor.processRequest()

Class Treasurer:

- If amount <= 500000:
 - Approve
- Else:
 - successor.processRequest()

Class ViceChancellor:

- If amount <= 1000000:
 - Approve
- Else:
 - Send to Board

Main:

- Create Chairman
- Create Treasurer
- Create ViceChancellor
- Chairman.setSuccessor(Treasurer)
- Treasurer.setSuccessor(ViceChancellor)
- purchase = Purchase(777, 350000, "Research")
- Chairman.processRequest(purchase)
- purchase = Purchase(888, 800000, "Construction")

7.7 Replacement Points

Chain of Responsibility can replace:

1. Large If-Else Logic

Instead of:

if chairman

else if treasurer

else if VC

use:

Chairman → Treasurer → VC